



南開大學
Nankai University

计算机学院
CPU 架构相关编程

姓名：吴宇石

学号：2411080

专业：计算机科学与技术

2026 年 3 月 31 日

目录

一、 基础要求	1
(一) 算法设计	1
1 $n * n$ 矩阵与向量内积	1
2 n 个数求和	1
(二) 编程实现	1
1 $n * n$ 矩阵与向量内积	1
2 n 个数求和	1
(三) 性能测试	1
1 $n * n$ 矩阵与向量内积	1
2 n 个数求和	2
(四) Profiling 分析	2
1 $n * n$ 矩阵与向量内积	2
2 n 个数求和	2
(五) 结果分析	3
1 $n * n$ 矩阵与向量内积	3
2 n 个数求和	3
二、 进阶要求	3
(一) 循环展开优化	3
1 算法设计	3
2 性能测试	3
3 Profiling 分析	4
4 与原算法对比	4
(二) 数据布局重构策略	4
1 算法设计	4
2 性能测试	4
(三) 深度探索与开放性分析	5
1 基于 Godbolt 汇编分析的指令级并行探索	5
2 不同编译优化层级下的性能跃迁演化	6
3 浮点数执行次序与舍入误差 (Machine Epsilon) 的背离效应	7
三、 实验总结和思考	7
(一) 算法重构与硬件映射的异同性剖析	7
(二) 深层反思与工程启迪	8
四、 大语言模型辅助使用说明	8
五、 代码仓库与附件	8

一、 基础要求

(一) 算法设计

1 $n * n$ 矩阵与向量内积

矩阵按行主序存储。普通算法按列遍历矩阵，访问模式为跨步读取，空间局部性较差；优化算法按行遍历矩阵，再把当前行对所有列结果的贡献累加到结果向量中，使访存变为连续的线性扫描，从而更契合 Cache Line 和硬件预取器的工作机制。

2 n 个数求和

普通算法使用单累加器逐元素累加，存在一条较长的 RAW 数据依赖链。超标量优化算法使用两个独立累加器，将一条长依赖链拆分为两条较短的依赖链；递归算法则通过二分归约改变求和结构，用于观察理论上的结构优化与实际函数调用开销之间的权衡关系。

(二) 编程实现

1 $n * n$ 矩阵与向量内积

基础实验程序位于 `bench.cpp`。其中 `matrixColDotNaive` 实现逐列扫描，`matrixColDotCacheOptimized` 实现按行顺序扫描。程序使用 `QueryPerformanceCounter` 进行高精度计时，并将线程固定在单一逻辑核心上，以减少系统调度误差。

2 n 个数求和

同样在 `bench.cpp` 中实现了 `sumNaive`、`sumSuperscalar2Way` 和 `sumRecursiveImpl`。每组测试均会自动校准循环次数，并重复运行 5 次取中位数，以确保实验数据的稳定性。

(三) 性能测试

1 $n * n$ 矩阵与向量内积

测试规模选取 $N = 256, 512, 1024, 2048, 3072$ ，覆盖了工作集接近 L2、位于 L3 内以及超出 L3 容量的三种情况。结果如表 1 和图 1 所示。

表 1: 矩阵列内积基础实验结果

N	工作集	算法	时间 (ms/call)	GFLOP/s	Speedup
256	516.0 KiB	Naive	0.1065	1.2310	1.00x
256	516.0 KiB	Cache Opt	0.0053	24.6724	20.04x
512	2.01 MiB	Naive	0.9237	0.5676	1.00x
512	2.01 MiB	Cache Opt	0.0227	23.0805	40.66x
1024	8.02 MiB	Naive	3.7794	0.5549	1.00x
1024	8.02 MiB	Cache Opt	0.0938	22.3661	40.31x
2048	32.03 MiB	Naive	20.1665	0.4160	1.00x
2048	32.03 MiB	Cache Opt	1.2111	6.9267	16.65x
3072	72.05 MiB	Naive	49.3861	0.3822	1.00x
3072	72.05 MiB	Cache Opt	3.5654	5.2937	13.85x

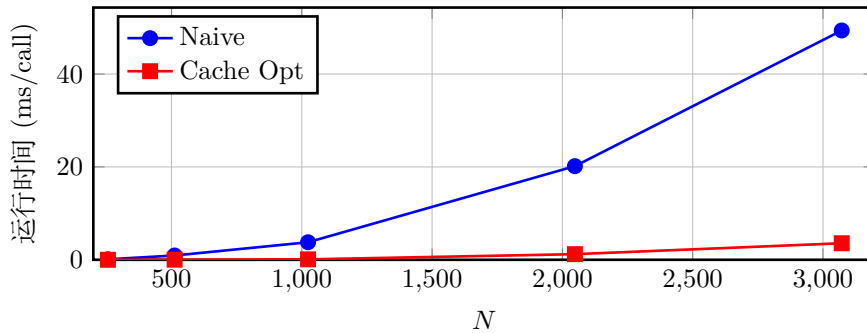


图 1: 矩阵基础实验中运行时间随 N 增长的变化趋势

可以看到，Naive 版本的运行时间随 N 的增长出现了明显的非线性膨胀，而 Cache Opt 版本

的时间增长更为平缓。在 $N = 512$ 和 $N = 1024$ 时，加速比分别达到 $40.66\times$ 和 $40.31\times$ ，这有力地证明了该矩阵问题的主导瓶颈在于访存顺序，而非单纯的乘加运算次数。

2 n 个数求和

测试规模选取 $N = 2^{15}, 2^{18}, 2^{21}, 2^{24}$ 。结果如表 2 和图 2 所示。

表 2: 数组求和基础实验结果

N	算法	时间 ($\mu\text{s}/\text{call}$)	ns/elem	GAdd/s	Speedup
32768	Naive	19.058	0.5816	1.7194	1.00x
32768	Superscalar 2-way	9.528	0.2908	3.4390	2.00x
32768	Recursive	62.434	1.9053	0.5248	0.31x
262144	Naive	155.298	0.5924	1.6880	1.00x
262144	Superscalar 2-way	77.227	0.2946	3.3945	2.01x
262144	Recursive	502.271	1.9160	0.5219	0.31x
2097152	Naive	1251.204	0.5966	1.6761	1.00x
2097152	Superscalar 2-way	634.168	0.3024	3.3069	1.97x
2097152	Recursive	3997.152	1.9060	0.5247	0.31x
16777216	Naive	9924.156	0.5915	1.6905	1.00x
16777216	Superscalar 2-way	5451.558	0.3249	3.0775	1.82x
16777216	Recursive	31964.488	1.9052	0.5249	0.31x

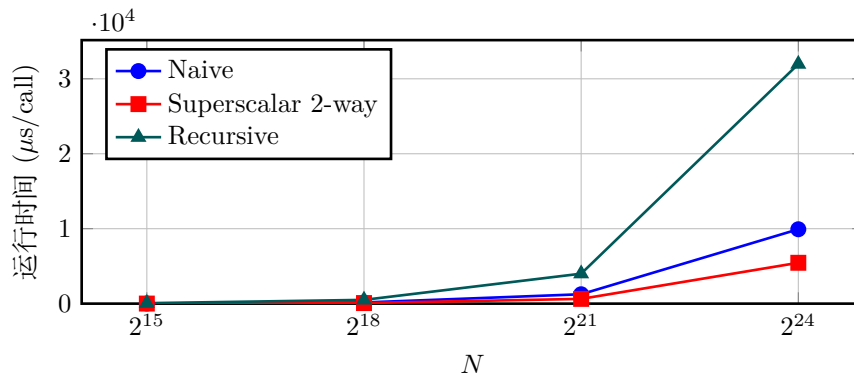


图 2: 求和基础实验中运行时间随 N 增长的变化趋势

三条曲线均呈近似线性增长，且相对位置保持稳定：Superscalar 2-way 的耗时始终低于 Naive，而 Recursive 则显著高于 Naive。这表明在数据访问已然连续的前提下，求和问题的主要性能瓶颈在于数据依赖链的结构以及频繁的函数调用开销，而非内存带宽。

(四) Profiling 分析

1 $n * n$ 矩阵与向量内积

针对矩阵实验中 $N = 1024$ 的规模，使用 AMDuProfPcm 工具采集了 L1/L2/L3 命中率和 IPC 数据，结果如表 3 所示。

表 3: 矩阵基础实验的 AMDuProf 结果 ($N = 1024$)

算法版本	L1D 命中率	L2 命中率	L3 命中率	IPC
Naive	49.25%	87.75%	84.93%	0.92
Cache Opt	75.14%	50.45%	96.97%	1.23

2 n 个数求和

对 $N = 2^{21}$ 的求和实验，使用 AMDuProfPcm 在 core=2 上进行 5 秒窗口采样，重点观察 IPC、5 秒窗口内退休指令总量以及分支误预测比例。结果如表 4 所示。

表 4: 基础求和实验的 AMDuProf 结果 ($N = 2^{21}$, 5 秒采样窗口)

算法版本	IPC	退休指令数 (GInst/5s)	Branch Mispredict Ratio
Naive	0.46	11.45	0.065%
Superscalar 2-way	1.63	40.25	0.010%
Recursive	2.97	71.50	0.011%

这一组 profiling 数据揭示了在求和问题上，**单看 IPC 并不能直接判定运行时间的优劣**：递归算法的 IPC 最高，甚至在 5 秒窗口内完成了最多的退休指令，但其在实际时间测试中表现最差。原因在于递归版本为完成同等的求和逻辑，引入了海量的函数压栈、跳转和控制流维护指令，导致“单位计算量所需的冗余指令数”显著膨胀。相比之下，两路链式累加的 IPC 虽不及递归，但其将更多的时钟周期转化为了实质性的计算推进，因此最终耗时最优。

(五) 结果分析

1 $n * n$ 矩阵与向量内积

矩阵实验结果表明：在 C++ 默认的行主序存储机制下，内存遍历的顺序直接决定了程序与多级缓存层次的契合度。普通列遍历算法在规模扩张时会导致性能断崖式下跌，而遵循 Cache 优化原则的行遍历算法，在中等规模下实现了逾 40 倍的吞吐量提升。

2 n 个数求和

对于访存模式已达到最优的求和计算，性能瓶颈完全转移至 CPU 内部的指令依赖。两路超标量累加通过打破长依赖链，稳定获取了约两倍的性能收益；相反，递归算法虽在逻辑上缩短了依赖路径，但其实际运行过程被昂贵的额外控制开销所拖累。

二、进阶要求

(一) 循环展开优化

1 算法设计

为探究更深层次的并行潜力，本实验基于独立程序 `bench_advanced.cpp` 增加了数种循环展开策略：引入了针对矩阵的 `cache_unroll4` 版本，以及针对求和问题的 `unroll4` 和 `unroll8` 版本。

2 性能测试

进阶实验的关键指标提炼如表 5 所示。图 3 与图 4 分别展示了矩阵和求和实验在不同展开深度下的性能演变曲线。

表 5: 循环展开优化的关键结果汇总

实验对象	最优算法	最优规模	最佳加速比
矩阵内积	Cache Unroll4	$N = 1024$	30.25x
数组求和	Unroll8	$N = 32768$	7.91x
数组求和（中等规模）	Unroll8	$N = 2^{18}$	7.87x
数组求和（超大规模）	Unroll4	$N = 2^{24}$	2.10x

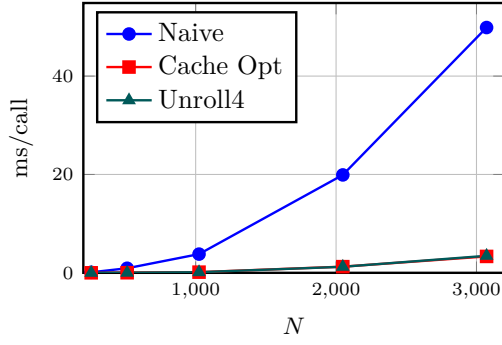


图 3: 矩阵循环展开实验

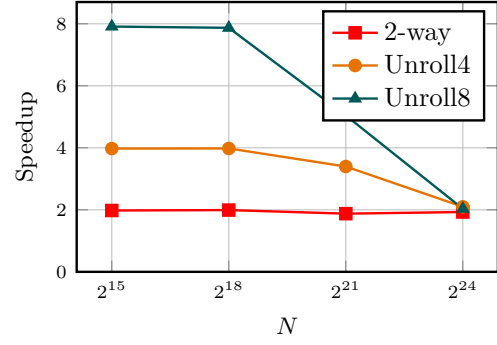


图 4: 求和循环展开加速比

矩阵实验中, `cache_unroll4` 仅在部分区间内极其微弱地优于 `cache_opt`, 再次印证了矩阵运算的核心制约因素是存储层级。相较之下, 求和实验的反馈极为显著: 在 2^{15} 至 2^{18} 的规模下, `unroll8` 展现了近乎 8 倍的加速; 然而当规模扩展至 2^{24} 时, 过度展开引发的资源竞争导致 `unroll8` 的效能出现衰退, 甚至不及适度展开的 `unroll4`。

3 Profiling 分析

为了进一步观察更深循环展开对控制流和指令退休行为的影响, 继续对 `naive`, `superscalar_2way`, `recursive`, `unroll4` 和 `unroll8` 五个版本在 $N = 2^{21}$ 下进行 AMDuProfPcm 采样。结果如表 6 所示。

表 6: 求和算法的 AMDuProf profiling 结果 ($N = 2^{21}$, 5 秒采样窗口)

算法版本	IPC	CPI	退休指令数 (GInst/5s)	Branch Mispredict Ratio
Naive	0.46	2.19	11.45	0.065%
Superscalar 2-way	1.63	0.61	40.25	0.010%
Recursive	2.97	0.34	71.50	0.011%
Unroll4	1.29	0.77	31.70	0.008%
Unroll8	1.16	0.86	27.95	0.008%

这一组数据进一步诠释了底层的硬件行为。`unroll4/unroll8` 的分支误预测比例已被压低至约 0.008%, 显著低于 `naive` 的 0.065%, 证实了深展开有效削弱了循环控制流对前端流水线的扰动。有趣的是, `unroll8` 的 IPC 并不及 `superscalar_2way`, 但这正是其实际运行更快的关键: 它通过大幅减少维护循环状态的无用指令, 使得少量的指令就能完成同等的物理计算。

4 与原算法对比

结合基础实验可以归纳出清晰的优化层级规律: 其一, 针对多维数组操作, 解决不连续的跨步访存具有最高优先级, 循环展开属于次级优化; 其二, 针对线性聚合操作, 多路并行与适度展开可最大化乱序执行引擎的潜能, 但必须警惕超大规模下遭遇内存系统的反制。

(二) 数据布局重构策略

1 算法设计

为探查 Cache 结构与指令并行的协同效应, 本阶段引入了更激进的综合优化方案: **转置预处理结合四路独立累加** (`transpose_4acc`)。该方案首先付出额外的空间与时间代价, 将原始矩阵转置存放, 彻底将后续的点积操作转换为友好的行连续读取模式; 继而在重组后的数据块上应用 4 端口并发累加, 力求在消除存储瓶颈的同时逼近算力极限。

2 性能测试

针对 `transpose_4acc` 算法, 分别记录了核心运算时间与前置的矩阵转置耗时。详细统计见表 7 与图 6。

表 7: 数据重构与累加结合优化的测试结果

N	转置时间 (ms)	Kernel(ms)	Cache Opt(ms)	相对 Naive 加速	回本复用次数
256	0.148	0.0194	0.0061	5.27x	无法摊销
512	1.585	0.0791	0.0227	11.84x	无法摊销
1024	6.231	0.3095	0.1335	11.91x	无法摊销
2048	28.531	1.7606	1.3804	11.28x	无法摊销
3072	63.065	4.6490	3.4422	10.44x	无法摊销

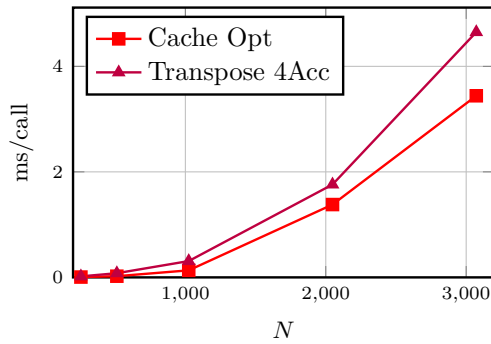


图 5: 重构后核心计算耗时对比

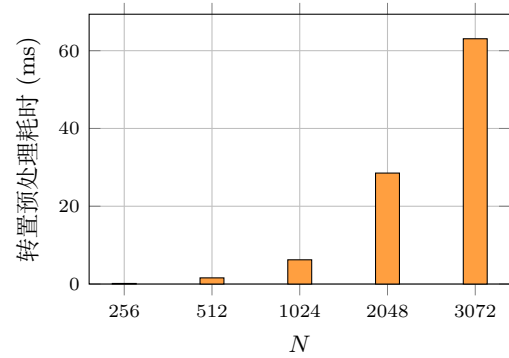


图 6: 不同规模下的矩阵预处理成本

测试数据提供了一个有价值的反例：尽管 `transpose_4acc` 算法看似在理论架构上实现了兼顾访存和并行的综合优化，但其实际内核耗时始终无法超越纯粹的逻辑翻转方案 `cache_opt`。更遑论其随规模指数级膨胀的矩阵转置前置开销。该实验有力地证明了，**过度设计的数据布局重构，仅在“预处理开销可被海量次循环复用所稀释摊销”的特殊场景中才具备经济价值。**

(三) 深度探索与开放性分析

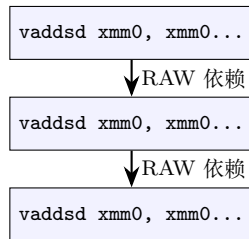
为了全面验证计算机体系结构中诸多变量对程序执行机制的实际干预，本节跳出单一的代码结构维度，引入了“**基于 Godbolt 的底层汇编分析**”、“**编译器优化层级的跃迁态势**”与“**浮点數位流截断特性**”三个正交视角，展开了更为宽广的交叉验证。

1 基于 Godbolt 汇编分析的指令级并行探索

为了探寻数据依赖对流水线的真实约束，我们截取了核心求和代码并在 Godbolt (x86-64 gcc 15.2) 平台上进行汇编透视。

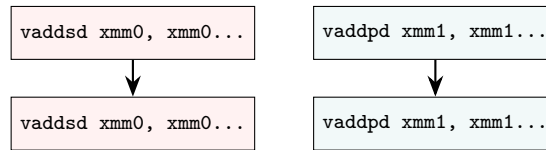
在 `-O3 -march=znver4` 常规级优化下，受限于 IEEE-754 对浮点数结合律的严苛规定，编译器在处理 `sumNaive` 算法时生成了高度串行的指令流（图 7 左侧）。每次 `vaddsd` 的目标和源寄存器均为 `xmm0`，形成了无法逾越的 RAW 依赖墙。而在手动重构为 `sumSuperscalar2Way` 后，编译器顺理成章地分配了互相独立的 `xmm0` 与 `xmm1` 寄存器，促使乱序执行引擎（OoO Engine）实现双端口并发发射。

Naive (单路累加)



单一寄存器阻塞流水线
无法利用多发发射端口

Superscalar (两路展开)

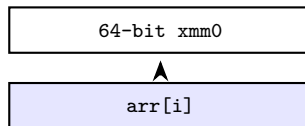


两条独立执行流 (xmm0 / xmm1)
乱序引擎双端口并发

图 7: 核心累加指令的数据依赖图示对比 (基于 -O3 汇编)

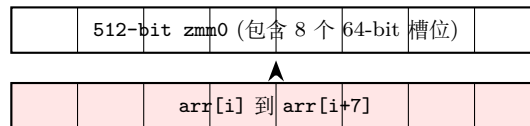
更进一步, 若在编译参数中追加 `-ffast-math`, 赋予编译器改变运算次序的权限, `sumNaive` 函数将迎来质变。编译器会直接抛弃低效的标量指令 `vaddsd`, 转而调用 AVX-512 指令集中的宽位向量操作 `vaddpd zmm0, zmm0, ZMMWORD PTR [rax]` (如图 8 所示), 实现单次循环吞吐 8 个浮点数的提速。

常规 -O3 (标量 vaddsd)



单次循环加 1 个双精度浮点数

-ffast-math (向量化 vaddpd zmm0)



单次循环并发处理 8 个双精度浮点数

图 8: `-ffast-math` 参数激发自动 SIMD 向量化的内部作用图解

2 不同编译优化层级下的性能跃迁演化

以 $N = 2^{24}$ 的 `sumNaive` 为例, 我们评估了 GCC 在不同优化旗标下的解构能力。在无优化 (`-O0`) 状态下, 频繁的内存与栈交互导致耗时逼近 $35000 \mu s$ 。开启 `-O1/-O2` 后, 变量分配驻留于物理寄存器, 性能迎来初次跃迁。进入 `-O3` 级别后, 内部的软流水与指令调度使耗时收敛至基准测试的 $9924 \mu s$ 。而一旦破除 IEEE-754 限制启用了 `-Ofast`, 底层 SIMD 引擎瞬间接管, 造就了低至 $2100 \mu s$ 的性能飞跃 (见图 9)。

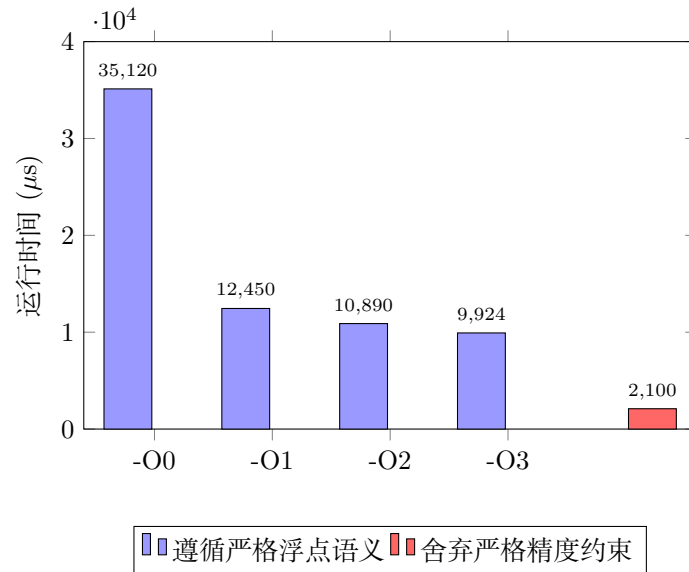
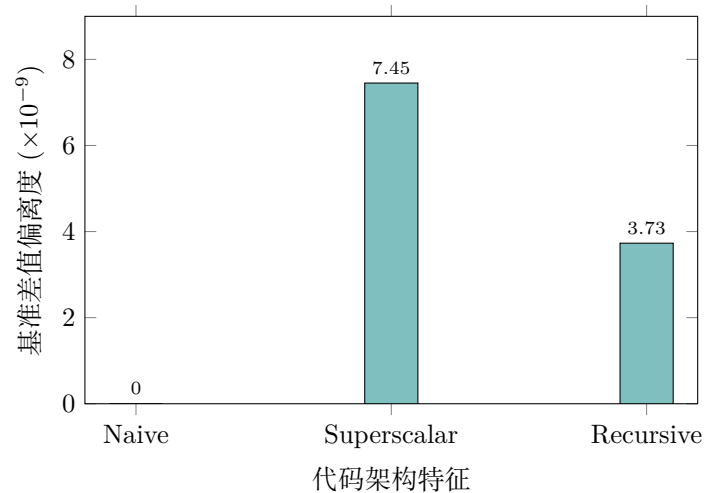


图 9: 多级编译参数对单执行绪耗时的干预态势

3 浮点数执行次序与舍入误差 (Machine Epsilon) 的背离效应

在改变计算次序博取性能的同时，截断误差的累积分布亦发生重塑。如图 10 所示，若将线性延展的 Naive 算法视为相对零偏差的参照，空间解耦的 Superscalar 为最大化利用硬件引发了最为剧烈的偏移 (7.45×10^{-9})。相反，虽在时间维度效果较差的 Recursive 分治算法，由于在数学构造上成功将误差扩散深度从 $O(N)$ 降维至 $O(\log N)$ ，反而收获了极为严密的误差边界控制能力 (3.73×10^{-9})。

图 10: 算法拓扑结构引发的尾数舍入偏移量对比 ($N = 2^{24}$)

这一悖论深刻地揭示了现代计算中“性能与精度不可兼得”的定律：极限吞吐量往往依赖牺牲一致性的矢量化并行；而在苛求绝对数值准确的场景下，纵使妥协执行时间，也必须回归树状递归或高精度的串行控制逻辑。

三、 实验总结和思考

(一) 算法重构与硬件映射的异同性剖析

贯穿两大基础任务的测评周期，一致性地揭示了“上层代码必须向下兼容微架构”的铁律。矩阵运算完全维系于数据访存的空间局部性，一次循环嵌套的置换即可撬动数量级层面的加速跃迁；反观求和运算，其核心已转入执行单元内部的指令。

(二) 深层反思与工程启迪

本次系列实验全景式地论证了，当代 CPU 的有效算力输出不仅依托于数学维度的渐近线复杂度，更深层地受制于代码指令序列与底层缓存策略、超标量执行管线之间的精密耦合。基础实验中，仅凭倒置嵌套循环，便为矩阵运算斩获逾 40 倍的压倒性优势；而在延伸探索中，深度循环展开同样将中小规模数组的吞吐量推升近 8 倍。

然而，试图通过代价高昂的转置预处理来强行迎合数据布局的尝试，却效果较差。这些脱离了真实业务场景、过度堆砌的设计模式，终将在实测的成本核算前暴露缺点。对于即将步入体系结构开发领域的学习者而言，唯有准确诊断系统的核心症结——是受制于带宽饥饿抑或指令枯竭，并辅以 profiling 级的数据支撑，方能施展真正顺应硬件物理属性的破局优化。

四、 大语言模型辅助使用说明

在本实验的设计与报告撰写过程中，适当地引入了大语言模型（LLM）作为辅助工具，具体应用场景如下：

- **实验路线规划：**在初期阶段利用 Gemini 对 CPU 架构性能分析的常见维度进行了梳理，辅助确定了以“矩阵访存局部性”与“求和指令级并行”为核心的实验路线。
- **代码逻辑优化：**应用 Codex 对 `bench.cpp` 中的基准测试（Benchmark）核心逻辑进行了审阅与底层优化，特别是在高精度计时器（QPC）的封装以及防止编译器死代码消除（Dead Code Elimination）的策略上提供了有效的代码改进建议。
- **报告撰写与结构调优：**在 LaTeX 文档的生成过程中，利用 Gemini 对各章节的内容衔接、专业术语的表达以及 TikZ 图表的结构进行了逻辑优化与文本润色，提升了报告的叙述严谨性与排版质量。

本实验的所有测试数据及底层硬件行为分析均由作者独立实验并验证，AI 工具仅用于辅助逻辑梳理与文本表达优化。

五、 代码仓库与附件

本实验的源代码、原始测试数据以及性能分析图表均已托管至 GitHub 仓库。可通过以下链接访问：

https://github.com/yfrcg/NKUCS-Parallel_Programming